



- ▶ **Lenguajes compilados e interpretados**
- ▶ **Entornos virtuales**
- ▶ **Ipython básico**

Bibliografía

- *Fundamentos de programación. sec. 1.8, 1.8.1 y 1.9.1-1.9.4*, Luis Joyanes.
- *Introducción a la programación con Python. sec 1.3 y cap. 2*, Andrés Marzal, Isabel Gracia y Pedro García.
- *Programming for computations - Python. sec. 2.1 y sec. 2.2*, Svein Lingen, Hans Petter Langtangen.
- *Python Software Foundation: venv — Creation of virtual environments*. Disponible en: <https://docs.python.org/3/library/venv.html>
- *Real Python. Python Virtual Environments: A Primer*. Disponible en :<https://realpython.com/python-virtual-environments-a-primer/>

Tipos de lenguajes

Anteriormente, vimos que las computadoras solo entienden el lenguaje binario, es decir, cadenas asociadas al alfabeto $\{0, 1\}$. En particular, trabajamos con cadenas de 8 bits (bytes). A este lenguaje se le conoce como *lenguaje de máquina*. Sin embargo, a la hora de dar instrucciones a la computadora, resulta engorroso y propenso a errores tener que hacerlo con esas cadenas interminables de ceros y unos.

Para solucionar esto, el primer paso histórico fue la creación del *lenguaje ensamblador (Assembly)*. En lugar de escribir secuencias binarias, este lenguaje utiliza nemónicos (instrucciones en texto corto como MOV, ADD o JMP) para representar directamente las operaciones del procesador.

Para que la computadora entienda estas instrucciones, se utiliza un **ensamblador (assembler)**: un programa traductor que convierte el código escrito en lenguaje ensamblador a su equivalente en lenguaje de máquina. Tanto el lenguaje de máquina como el ensamblador son considerados *lenguajes de bajo nivel*, pues están directamente atados a la arquitectura física del procesador (por ejemplo, x86, ARM) y requieren gestionar manualmente los registros de la CPU y las direcciones de memoria.

Con el tiempo, para abstraerse del hardware y escribir programas portátiles y legibles, se crearon los *lenguajes de alto nivel*. Existen numerosos lenguajes de alto nivel: Python, Java, C, C++, JavaScript, Ruby, Swift, Visual Basic, Perl, .NET, Fortran, Pascal, CUDA, entre otros. Aunque difieren en muchos detalles, todos comparten la propiedad de tener sintaxis y semántica bien definidas y no ambiguas.

Dado que la computadora solo comprende el lenguaje de bajo nivel, cuando escribimos un programa en un lenguaje de alto nivel debemos usar un traductor para convertirlo a lenguaje de máquina. Esto se realiza principalmente mediante dos esquemas: *compilación e interpretación*.

- **Lenguajes Compilados (ej. C, C++, Fortran):** El programa fuente se pasa por un software llamado *compilador* (gcc, gfortran), el cual analiza todo el código y genera un archivo ejecutable binario independiente (código de máquina nativo para una arquitectura específica).
 - *Ventaja:* Ejecución extremadamente rápida y optimizada para el procesador.
 - *Desventaja:* Si se cambia el sistema operativo o la arquitectura, se debe recompilar. Los errores se detectan antes de la ejecución.
- **Lenguajes Interpretados (ej. Ruby, Python, JavaScript):** Un programa denominado *intérprete* lee el código fuente línea por línea y lo traduce e instruye al procesador en tiempo real.
 - *Ventaja:* Alta portabilidad y facilidad de depuración durante el desarrollo.
 - *Desventaja:* Menor velocidad de ejecución, ya que el proceso de traducción ocurre mientras el programa corre.

El modelo híbrido de Python

A diferencia de los lenguajes interpretados puros, Python no lee directamente el texto del código fuente en cada ejecución. En su lugar, utiliza un esquema de dos pasos:

1. Traduce el archivo fuente (.py) a un código intermedio ejecutable llamado *Bytecode* (.pyc).
2. La máquina virtual de Python (PVM) procesa y ejecuta dicho *Bytecode*.

Este enfoque permite combinar la flexibilidad de los lenguajes interpretados con una mejor velocidad al reejecutar módulos ya procesados.

Hidden Figures

La película *Talentos Ocultos* (*Hidden Figures*, 2016) ilustra perfectamente la transición del cálculo manual a la programación en la NASA durante los años 60. En ella se destaca el papel de Dorothy Vaughan, quien aprende de manera autodidacta **Fortran** (FORmula TRANslator) —uno de los primeros lenguajes de alto nivel— para programar la supercomputadora IBM 7090 y capacitar a su equipo.

Más allá del reto técnico de pasar del cálculo manual a la programación en la IBM 7090 usando Fortran, la película muestra cómo mujeres afroamericanas en la NASA de los años 60, enfrentaron la intersección del machismo estructural y el racismo de la segregación (espacios, baños y herramientas de trabajo "solo para blancos"). Su historia refleja la batalla que se debió hacer (y que todavía se debe hacer) contra la exclusión social en cualquier ámbito y en particular las ciencias exactas y la computación.

¿Por qué Python?

Si existen tantos lenguajes de programación y, además, los que son compilados como C, C++ o Fortran son más rápidos, ¿por qué aprender Python? Hay diferentes buenas respuestas para esto, un par radican en el hecho de que su sintaxis es simple, el software es libre y la comunidad que lo mantiene es muy grande. Así mismo, una serie de ventajas que presenta Python son:

1. Python permite expresar conceptos matemáticos y físicos complejos con muy pocas líneas de código. Su sintaxis es legible y carece de la sobrecarga de gestión manual de memoria (como punteros o asignación explícita) presente en lenguajes como C++. Esto permite a concentrarse en la *física del problema* y no en el código.
2. Python cuenta con un conjunto de librerías especializadas que cubren prácticamente cualquier necesidad del análisis numérico y la modelación:
 - **NumPy**: Manejo eficiente de vectores, matrices y álgebra lineal.
 - **SciPy**: Solución de ecuaciones diferenciales, integración numérica, optimización y transformación de Fourier.
 - **Matplotlib / Seaborn**: Visualización de datos, generación de gráficos de calidad de publicación.
 - **SymPy**: Cálculo simbólico (álgebra, derivadas e integrales analíticas).
 - **Pandas**: Manejo y estructuración de grandes volúmenes de datos experimentales.
3. Herramientas como **IPython** y las **Jupyter Notebooks** permiten mezclar texto ejecutable, ecuaciones en $\text{\LaTeX}$, gráficos en tiempo real y código interactivo.

Entornos virtuales

Antes de comenzar a trabajar con Python, es recomendable crear un entorno virtual para el proyecto que deseamos realizar. Un entorno virtual es una copia aislada de Python dentro de un proyecto que permite instalar librerías sin afectar al Python del sistema. El entorno virtual permite aislar las dependencias y librerías necesarias para cada proyecto, evitando conflictos con otras instalaciones de Python en el sistema. Además, facilita la gestión de versiones y la reproducibilidad del entorno de desarrollo.

Para crear un entorno virtual, se ejecuta el siguiente comando en la terminal:

```
1 python3 -m venv .venv
```

`python3` usa el intérprete Python 3 instalado en el sistema. El comando `-m venv` indica que se desea crear un entorno virtual, y `.venv` es el nombre del directorio donde se almacenará el entorno virtual (el `.` al inicio indica que es un directorio oculto). Confirmamos que quedó instalado el entorno virtual con el comando `ls -la`. Ingresamos al directorio `cd .venv/` y ejecutamos el comando `ls -la` para ver su contenido. Allí encontrarás los directorios `bin`, `include`, `lib` y el archivo `pyvenv.cfg`. En `bin` se encuentran los ejecutables de Python y `pip`, en `lib` se almacenan las librerías instaladas y se muestra la versión de Python utilizada. En `include` se encuentran los archivos de cabecera de Python y el archivo `pyvenv.cfg` contiene la configuración del entorno virtual.

Una vez creado el entorno virtual, es necesario activarlo para poder utilizarlo. Se ejecuta el siguiente comando en la terminal:

```
1 source .venv/bin/activate
```

Note que el prompt de la terminal cambia, indicando que el entorno virtual está activo. Una vez activo el entorno virtual, se pueden instalar las librerías necesarias para el proyecto utilizando pip.

```
1 pip install --upgrade pip
2 pip install numpy scipy matplotlib sympy pandas jupyter tqdm
```

Luego guardamos las dependencias del proyecto en un archivo requirements.txt con el comando: `pip freeze > requirements.txt`. Si queremos recrear el entorno en otra máquina, podemos instalar las dependencias desde el archivo requirements.txt con el comando: `pip install -r requirements.txt`. Para desactivar el entorno virtual, se ejecuta el comando `deactivate`. Para el elinir el entorno virtual, se ejecuta el comando `rm -rf .venv`. **Es importante no sincronizar el entorno virtual con el repositorio de git, para ello se recomienda agregar el directorio .venv al archivo .gitignore.**

No usar sudo

Dentro del entorno, nunca es necesario usar privilegios de root.

IPython

Llegó la hora de comenzar a trabajar con python. Normalmente en la consola podemos escribir `python3` y se abre el intérprete de python. Sin embargo, es más recomendable usar el intérprete IPython, que es una versión mejorada del intérprete de Python. Para abrir el intérprete de IPython, se ejecuta el comando `ipython3` en la terminal. Se puede salir del intérprete de IPython con el comando `exit` o `Ctrl+D`. En IPython se pueden ejecutar comandos del sistema operativo utilizando el prefijo `!`. Por ejemplo, `!ls` lista los archivos o compilar un .tex con `!pdflatex archivo.tex`. Al ejecutarlo, podemos usarlo como una calculadora, por ejemplo:

```
1 Python 3.14.6 (main, Jun 11 2026, 00:00:00) [GCC 16.1.1 20260515 (Red Hat
   16.1.1-2)]
2 Type 'copyright', 'credits' or 'license' for more information
3 IPython 9.15.0 -- An enhanced Interactive Python. Type '?' for help.
4 Tip: Use the IPython.lib.demo.Demo class to load any Python script as an
   interactive demo.
5
6 In [1]: 2+2
7 Out[1]: 4
8
9 In [2]: 2*3
10 Out[2]: 6
11
12 In [3]: 20/2
13 Out[3]: 10.0
14
15 In [4]: 3**2
16 Out[4]: 9
17
18 In [5]: 1-2+3
19 Out[5]: 2
```

```
20
21 In [5]: 1-(2+3)
22 Out[5]: -4
23
24 In [67]: a = 3      # defino a
25
26 In [68]: a += 5     #incremento a en 5
27
28 In [69]: a
29 Out[69]: 8
30
31 In [70]: b = 6      # defino b
32
33 In [71]: b -=1      #le resto 1
34
35 In [72]: b
36 Out[72]: 5
```

Además de su uso como calculadora, IPython tiene **Autocompletado con Tab**: Al escribir las primeras letras de una variable, función o módulo, presionar Tab despliega las opciones disponibles. **Introspección y Ayuda (? y ??)**: Agregar un signo de interrogación al final de cualquier función muestra su documentación (*docstring*). Con dos signos (??) se intenta mostrar el código fuente interno.

Precedencia de Operadores

Al evaluar expresiones matemáticas, Python sigue el orden estándar de las operaciones algebraicas (jerarquía de operaciones). Las operaciones de igual jerarquía (como suma y resta) se evalúan de izquierda a derecha:

$$1 - (2 + 3) = 1 - 5 = -4 \quad \text{frente a} \quad 1 - 2 + 3 = -1 + 3 = 2$$

Sin embargo, la multiplicación (*) y la división (/) tienen mayor precedencia que la suma (+) y la resta (-). Para alterar el orden natural de evaluación deben utilizarse paréntesis ():

```
1 In [1]: 2 * 5 + 3
2 Out[1]: 13
3
4 In [2]: 2 + 5 * 3
5 Out[2]: 17
6
7 In [3]: (2 + 5) * 3
8 Out[3]: 21
```

Resumen de jerarquía (de mayor a menor prioridad):

1. **Paréntesis ()**: Rompen la jerarquía por defecto.
2. **Potenciación ****: Se evalúa de **derecha a izquierda** ($2^{3^2} = 2^9$).
3. **Multiplicación *, División /, División entera //** y **Módulo %**: Evaluadas de izquierda a derecha.

4. **Suma + y Resta -**: Mención de menor prioridad, evaluadas de izquierda a derecha.

```
1 In [4]: 2 ** 3 ** 2      # Equivale a 2**(3**2) = 2**9
2 Out[4]: 512
3
4 In [5]: (2 ** 3) ** 2    # Fuerzado con parentesis = 8**2
5 Out[5]: 64
```

Tipos de datos

Los tipos de datos en python se pueden ver con la función `type`. Note que por definición, la operación entre un entero y un flotante devuelve un flotante. Además, la definición de un número complejo $z = 3 + 5i$ se hace mediante el símbolo j $z = 3 + 5j$ con el símbolo j pegado al número. También puede hacerse con la función `z = complex(3, 5)`. Tanto la parte real como la parte imaginaria son tomadas como floats.

```
1 In [8]: x = 2                      # Entero (int)
2
3 In [13]: type(x)
4 Out[13]: int
5
6 In [3]: y=2.                      # Flotante (float)
7
8 In [20]: type(y)
9 Out[20]: float
10
11 In [21]: w = x+y
12 In [22]: type(w)
13 Out[22]: float
14
15 In [40]: z = 3 + 5j                # Complejo (complex)
16 In [40]: type(z.real)
17 Out[40]: float
18
19 In [41]: type(z.imag)
20 Out[41]: float
21
22 In [17]: type(True)                # booleano (bool)
23 Out[17]: bool
24
25 In [42]: s= "Sociedad insociable"  # Cadena (str)
26
27 In [43]: type(s)
28 Out[43]: str
29
30 In [47]: 4*s
31 Out[47]: 'Sociedad insociableSociedad insociableSociedad insociableSociedad
    insociable'
32
33 In [1]: lista = [1,1,2,3,5,8,13,21] # Lista (list)
```

```
34
35 In [2]: type(lista)
36 Out[2]: list
37
38 In [23]: sum(lista)
39 Out[23]: 54
40
41 In [24]: max(lista)
42 Out[24]: 21
43
44 In [25]: min(lista)
45 Out[25]: 1
46
47 In [6]: asistentes = ["Jennifer", "Jorge", "Saray", "Santiago", "Felipe", "
    David"]
48
49 In [7]: type(asistentes) #lista de cadenas
50 Out[7]: list
51
52 In [8]: len(asistentes)
53 Out[8]: 6
54
55 In [9]: asistentes[0] #note que comienza a contar desde cero
56 Out[9]: 'Jennifer'
57
58 In [12]: asistentes[5]
59 Out[12]: 'David'
60
61 In [14]: asistentes[-1] #ver última entrada
62 Out[14]: 'David'
63
64 In [15]: asistentes[-2] #ver penúltima entrada
65 Out[15]: 'Felipe'
66
67 In [13]: asistentes[6] #error no existe la entrada 6
68 -----
69 IndexError                                Traceback (most recent call last)
70 Cell In[13], line 1
71 ----> 1 asistentes[6]
72
73 IndexError: list index out of range
74
75 In [18]: asistentes[0:3] #asistentes desde el cero al 3 (no incluye al 3)
76 Out[18]: ['Jennifer', 'Jorge', 'Saray']
77
78 In [19]: asistentes[:3] #desde inicio hasta el 3 (sin incluir al 3)
79 Out[19]: ['Jennifer', 'Jorge', 'Saray']
80
81 In [20]: asistentes[3:] #desde el 3 (incluido) hasta el final
82 Out[20]: ['Santiago', 'Felipe', 'David']
```

Funciones matemáticas integradas.

Python incluye por defecto varias funciones útiles para manipular números:

```
1 In [19]: abs(-15.4)          # Valor absoluto / Magnitud
2 Out[19]: 15.4
3
4 In [20]: round(3.14159, 2) # Redondeo a N decimales
5 Out[20]: 3.14
6
7 In [21]: pow(2, 4)           # Equivalente a 2**4
8 Out[21]: 16
9
10 In [22]: min(5, 2, 9, -1)  # Valor minimo
11 Out[22]: -1
12
13 In [23]: max(5, 2, 9, -1)  # Valor maximo
14 Out[23]: 9
```

Salida de datos con print() y cadenas formateadas (f-strings):

Para mostrar resultados en pantalla se utiliza la función `print()`. En física es común trabajar con cantidades muy grandes o muy pequeñas, por lo que Python permite definir números directamente en notación científica utilizando la letra `e` (o `E`) para representar la potencia de 10. Por ejemplo, definamos la velocidad de la luz en el vacío $c \approx 2,99792458 \times 10^8$ m/s:

```
1 In [31]: c = 2.99792458e8 # m/s
2
3 In [32]: print(c)
4 299792458.0
5
6 In [33]: print("La velocidad de la luz en el vacío es:", c, "m/s")
7 La velocidad de la luz en el vacío es: 299792458.0 m/s
```

La forma moderna y recomendada en Python para combinar texto y variables es mediante las *f-strings* (anteponiendo la letra `f` antes de las comillas). Esto, permite evaluar variables y expresiones directamente dentro de llaves `{}`. Es decir, todo lo pongas dentro de las llaves `{}` es reconocido como código por python.

```
1 In [34]: print(f"La velocidad de la luz es c = {c} m/s")
2 La velocidad de la luz es c = 299792458.0 m/s
```

Además, podemos aplicar especificadores de formato dentro de la llave utilizando dos puntos :

```
1 # Formato en notacion cientifica con 2 decimales (:.2e)
2 In [35]: print(f"Notacion cientifica: c = {c:.2e} m/s")
3 Notacion cientifica: c = 3.00e+08 m/s
4
5 In [54]: print(f"Notacion cientifica: c = {c:.2E} m/s")
6 Notacion cientifica: c = 3.00E+08 m/s
7
8 In [57]: print(f"Notacion cientifica: c = {c:.3E} m/s")
```

```
9 Notacion cientifica: c = 2.998E+08 m/s
10
11 # Formato en punto flotante con 2 decimales (:.2f)
12 In [36]: print(f"Notacion flotante: c = {c:.2f} m/s")
13 Notacion flotante: c = 299792458.00 m/s
14
15 # Formato con separadores de miles y 2 decimales (:,.2f)
16 In [37]: print(f"Con separadores: c = {c:,.2f} m/s")
17 Con separadores: c = 299,792,458.00 m/s
```

Dentro de las llaves no solo va el nombre de las variables que queremos mostrar, también podemos hacer operaciones

```
1 In [4]: print(f"El doble de c es {2 * c} m/s")
2 El doble de c es 599580000.0 m/s
3
4 In [5]: print(f"La luz recorre {c / 1000} km en un segundo")
5 La luz recorre 299790.0 km en un segundo
```

También puedes usar cadenas

```
1 In [6]: materia = "física computacional"
2
3 In [7]: print(f"Bienvenidos a {materia.upper()}")
4 Bienvenidos a FÍSICA COMPUTACIONAL
```

Importación de módulos

Las funciones predeterminadas en Python son limitadas. Por ejemplo, por defecto no podemos calcular razones trigonométricas (como seno o coseno), logaritmos ni raíces cuadradas. Para ello existen los **módulos** (o librerías), los cuales no se cargan por defecto en memoria para optimizar recursos.

El módulo estándar para operaciones matemáticas es *math*. Existen varias formas de importarlo:

1. **Importación específica:** `from math import sin`
Importa solo la función solicitada. Puede resultar engorroso si necesitamos usar decenas de funciones distintas.
2. **Importación masiva (no recomendada):** `from math import *`
Importa todas las funciones del módulo. El problema de esta práctica (llamada *contaminación del espacio de nombres*) es que si previamente habíamos definido una variable llamada *sin*, el módulo la sobrescribirá sin advertencia.
3. **Importación mediante espacio de nombres (recomendada):**
Es preferible usar `import math` o asignarle un alias corto como `import math as mate`.

De esta forma, las funciones se invocan de manera segura usando la notación de punto. Note que los ángulos deben ingresarse siempre en radianes:

```
1 In [38]: import math
2
```

```

3 # Pasar 90 grados a radianes: (90 * pi / 180)
4 In [39]: math.sin(90 * math.pi / 180)
5 Out[39]: 1.0
6
7 In [40]: import math as mate
8
9 In [41]: mate.sin(mate.pi / 2)
10 Out[41]: 1.0
11
12 # Tambien se puede usar la funcion integrada math.radians():
13 In [42]: math.sin(math.radians(90))
14 Out[42]: 1.0

```

En la siguiente tabla se muestran algunas funciones del módulo math.

Comando en Python	Descripción / Uso	Expresión Matemática
1. Trigonometría y Ángulos		
math.sin(x), cos(x), tan(x)	Seno, coseno y tangente (en radianes)	$\sin(x)$, $\cos(x)$, $\tan(x)$
math.asin(x), acos(x), atan(x)	Funciones trigonométricas inversas	$\arcsin(x)$, $\arccos(x)$, $\arctan(x)$
math.radians(x) / degrees(x)	Conversión entre grados y radianes	$\theta_{\text{rad}} \leftrightarrow \theta_{\text{deg}}$
2. Exponenciales, Logaritmos y Raíces		
math.exp(x)	Exponencial	e^x
math.log(x)	Logaritmo natural	$\ln(x)$
math.log10(x)	Logaritmo en base 10	$\log_{10}(x)$
math.sqrt(x)	Raíz cuadrada	$\sqrt{x}$
3. Redondeo y Constantes		
math.floor(x) / ceil(x)	Redondeo hacia abajo (piso) / arriba (techo)	$\lfloor x \rfloor$, $\lceil x \rceil$
math.pi	Constante Pi	$\pi \approx 3,14159$
math.e	Base del logaritmo natural	$e \approx 2,71828$

Representación y límites del Número de Máquina

Como vimos en las clases pasadas, es crucial entender que las computadoras no trabajan con números reales arbitrarios, sino con una aproximación finita, *puntos flotantes* (estándar IEEE 754 de 64 bits). Podemos inspeccionar los límites físicos de representación numérica de nuestro procesador directamente en IPython:

```

1 In [10]: import sys
2
3 In [11]: sys.float_info.epsilon # Precision de maquina (\epsilon_mach)
4 Out[11]: 2.220446049250313e-16
5
6 In [12]: sys.float_info.max      # Límite de desbordamiento (Overflow)
7 Out[12]: 1.7976931348623157e+308
8
9 In [13]: sys.float_info.min      # Límite de subdesbordamiento (Underflow)
10 Out[13]: 2.220446049250313e-308

```

- **Precisión de máquina** ($\epsilon_{mach} \approx 2,22 \times 10^{-16}$): Es la distancia entre el número 1,0 y el siguiente flotante representable. Determina que en precisión doble tenemos aproximadamente 15 a 17 dígitos decimales de precisión.
- **Efectos prácticos:** Operaciones como $1,0 + 10^{-16}$ resultarán exactamente en 1,0 en la máquina debido a la pérdida de significancia.

Problemas:

Resuelva los siguientes ejercicios ejecutando los comandos en su sesión de IPython y registrando las salidas correspondientes:

1. Notación científica y f-strings (Tiempo que tarda la luz del Sol a la Tierra):

La distancia media de la Tierra al Sol es $d = 1,496 \times 10^{11}$ m y la velocidad de la luz es $c = 2,99792458 \times 10^8$ m/s. Define ambas variables en notación científica (1,496e11 y 2,99792458e8). Calcula el tiempo $t = d/c$ en segundos e imprímelo en pantalla usando una *f-string* con:

- Formato flotante tradicional con 2 decimales (: .2f).
- Formato en notación científica con 3 decimales (: .3e).

2. Números complejos y magnitud:

En el estudio de circuitos de corriente alterna, la impedancia de un componente viene dada por $Z = 4 + 3j \Omega$.

- Define Z en IPython y muestra su parte real y su parte imaginaria mediante sus atributos `.real` y `.imag`.
- Calcula la magnitud $|Z|$ usando la función integrada `abs(Z)`.
- Verifica que el resultado coincide calculando $\sqrt{\text{real}^2 + \text{imag}^2}$ usando el módulo `math`.

3. Trigonometría y conversión de ángulos (Alcance de un proyectil):

El alcance horizontal de un proyectil disparado con velocidad v_0 y ángulo θ respecto a la horizontal es:

$$R = \frac{v_0^2 \sin(2\theta)}{g}$$

Para $v_0 = 35$ m/s, $\theta = 30^\circ$ y $g = 9,81$ m/s²:

- Importa el módulo `math` y convierte θ a radianes usando `math.radians()`.
- Calcula el alcance R e imprime el resultado redondeado a 2 decimales.

4. Exponenciales y descomposición radiactiva:

La cantidad de núcleos radiactivos restantes en una muestra sigue la ley $N(t) = N_0 e^{-\lambda t}$. Para una muestra con $N_0 = 5000$ núcleos, tasa de decaimiento $\lambda = 0,035$ s⁻¹ y un tiempo transcurrido de $t = 12$ s:

- Calcula $N(12)$ usando `math.exp()`.
- Redondea el resultado al número entero más cercano hacia abajo usando `math.floor()`.

5. Logaritmos e intensidad sonora (Decibeles):

El nivel de intensidad sonora en decibeles (dB) se define como:

$$\beta = 10 \log_{10} \left(\frac{I}{I_0} \right)$$

donde $I_0 = 10^{-12} \text{ W/m}^2$ es el umbral de audición. Si una fuente produce una intensidad $I = 2,5 \times 10^{-4} \text{ W/m}^2$, calcula el nivel de intensidad β usando `math.log10()` y exprésalo con una *f-string*.

6. Precedencia de operadores y asociatividad de exponentes:

Sin usar Python inicialmente, prediga el resultado de las siguientes expresiones. Luego, ejecútelas en IPython para verificar sus respuestas y explique las diferencias:

a) `4 ** 3 ** 2` frente a `(4 ** 3) ** 2`

b) `10 + 20 * 3 / 5 - 2`

c) `10 + 20 * (3 / (5 - 2))`

7. Identidad trigonométrica fundamental:

Para un ángulo arbitrario $\theta = 75^\circ$, verifique en IPython la validez numérica de la identidad fundamental $\sin^2(\theta) + \cos^2(\theta) = 1$. *Nota:* Recuerde convertir el ángulo a radianes antes de evaluar las funciones.